# Lecture 1: Analysis of Algorithms

CSI2110 – Data Structures and Algorithms

Lecture study notes

**Source document:** `../lec01-analysisofalgorithms.pdf`. These notes synthesize the source and add worked explanations. Page references use PDF page numbers, which can differ from printed slide numbers. Pseudocode is normalized for readability and boundary cases.

## Contents


**1 Algorithms and efficiency** **2**

**2 Pseudocode and the RAM model** **2**

2.1 Worked example: finding the maximum . . . . . 2

**3 Asymptotic notation** **2**

3.1 Proofs by definition . . . . . 3

3.2 Rules and growth order . . . . . 3

3.3 Lecture classification exercise . . . . . 4

**4 Analyzing loops** **4**

**5 Prefix averages: avoiding repeated work** **4**

**6 Prefix averages at geometric steps** **5**

**7 Mathematical toolkit** **5**

# 1 Algorithms and efficiency

*Source: PDF pages 4–14.* An algorithm is a finite, well-defined procedure for solving a problem. Evaluate **correctness**, **running time**, and **memory usage**. The input size $n$ measures the amount of data; for an array it is usually the number of entries.

Different inputs of the same size can have different costs. Linear search may find a target immediately, at the last index, or not at all. Its best-case time is $\Theta(1)$ and worst-case time is $\Theta(n)$.

Experimental analysis implements and times a program on varied inputs. It reveals practical performance, but depends on hardware, language, compiler, and the chosen inputs. Theoretical analysis counts operations independently of a particular machine.

# 2 Pseudocode and the RAM model

*Source: PDF pages 15–19.* Pseudocode expresses an algorithm without unnecessary implementation details. Indentation defines blocks; assignment is written $\leftarrow$, while $=$ means equality.

The random access machine (RAM) model assumes constant-time memory access and constant-time primitive operations such as assignment, arithmetic on machine-sized values, comparisons, and return. A loop or a method call is not automatically one primitive operation: count the work it performs.

## 2.1 Worked example: finding the maximum

Assume $n \geq 1$.

```
arrayMax(data):
    currentMax = data[0]
    for i = 1 to data.length - 1:
        if data[i] > currentMax:
            currentMax = data[i]
    return currentMax
```

*Source: PDF pages 19–31.* There are always $n-1$ element comparisons. The current maximum is assigned once initially and then at most $n-1$ more times. The lecture's particular counting convention gives $4n+3$ primitive operations in the best case and $6n+1$ in the worst case. Exact constants depend on the counting convention; both cases take $\Theta(n)$ time.

**Invariant:** before iteration $i$, `currentMax` is the maximum of entries $0, \ldots, i-1$. Comparing entry $i$ maintains the invariant. At termination all entries have been considered.

# 3 Asymptotic notation

*Source: PDF pages 33–54,62–68.* Asymptotic analysis describes growth for sufficiently large $n$. Assume the compared functions are eventually nonnegative and $g(n)$ is eventually positive.

$$
\begin{aligned}
f \in O(g) &\iff \exists c > 0, n_0 : \quad f(n) \leq cg(n) \quad \forall n \geq n_0, \\
f \in \Omega(g) &\iff \exists c > 0, n_0 : \quad f(n) \geq cg(n) \quad \forall n \geq n_0, \\
f \in \Theta(g) &\iff \exists c_0, c_1 > 0, n_0 : \quad c_0 g(n) \leq f(n) \leq c_1 g(n) \quad \forall n \geq n_0.
\end{aligned}
$$

- $O$ is an upper bound, $\Omega$ a lower bound, and $\Theta$ a tight bound.
- $c$ absorbs constant factors; $n_0$ excludes a finite initial range.

- $c$ and $n_0$ must be constants independent of $n$. They need not be the smallest possible values.
- $\Theta(g) = O(g) \cap \Omega(g)$ and $f \in O(g)$ iff $g \in \Omega(f)$.

**Key idea.** $O$ does not mean "worst case" and $\Omega$ does not mean "best case." First choose the running-time function (best, worst, or average case), then bound that function.

### 3.1 Proofs by definition

For $n \geq 1$,

$$25n + 2110 \leq 25n + 2110n = 2135n.$$

Thus $25n + 2110 \in O(n)$ with $c = 2135, n_0 = 1$.

For $n \geq 1$,

$$3n^3 + 20n^2 + 5 \leq 28n^3.$$

Hence the polynomial is $O(n^3)$. It is also $\Omega(n^3)$ because it is at least $3n^3$, so it is $\Theta(n^3)$.

For $n \geq 1$,

$$20n^2 \leq 20n^2 + 4n \leq 24n^2,$$

proving $\Theta(n^2)$ with $c_0 = 20, c_1 = 24, n_0 = 1$.

For base-2 logarithms and $n \geq 2$, $\log_2 n \geq 1$, so

$$n \log_2 n \leq n \log_2 n + 204n \leq 205n \log_2 n.$$

This proves $\Theta(n \log n)$. The threshold $n_0 = 2$ is needed for this upper bound; $n = 1$ does not satisfy it.

### 3.2 Rules and growth order

For nonnegative functions:

- Reflexivity: $f \in O(f)$; transitivity: $f \in O(g), g \in O(h) \Rightarrow f \in O(h)$.
- If $f, g \in O(h)$, then $f + g \in O(h)$.
- If $f_1 \in O(f_2)$ and $g_1 \in O(g_2)$, then $f_1 g_1 \in O(f_2 g_2)$.
- $\max\{f, g\} \leq f + g \leq 2 \max\{f, g\}$, so $f + g \in \Theta(\max\{f, g\})$.

Drop constant factors and lower-order terms when finding a tight class. An eventually nonnegative degree-$d$ polynomial with positive leading coefficient is $\Theta(n^d)$. Using absolute coefficient values gives a valid upper-bound proof even when some coefficients are negative.

Common growth rates, in increasing order, are

$$1, \quad \log n, \quad n, \quad n \log n, \quad n^2, \quad n^3, \quad 2^n, \quad n2^n, \quad e^n, \quad n!.$$

A fixed logarithm base greater than one changes only a constant factor. A fixed exponential base *can* change the growth class: $3^n \notin O(2^n)$.

### 3.3 Lecture classification exercise

*Source: PDF pages 66.*

| Function | $O(n^3)$ | $\Omega(n^3)$ | $\Theta(n^3)$ |
|---|---|---|---|
| $77n^2$ | Yes | No | No |
| $6n^3 + n^2 \log_2 n$ | Yes | Yes | Yes |
| $11n^4$ | No | Yes | No |
| $n^2$ if even; $n^4$ if odd | No | No | No |

The last function violates the upper bound on arbitrarily large odd inputs and the lower bound on arbitrarily large even inputs. Asymptotic definitions require the inequality for *all* sufficiently large inputs.

## 4 Analyzing loops

*Source: PDF pages 56–60.* Count iterations, multiply by the work per iteration, and sum costs when bounds depend on an outer variable.

| Pattern, with constant work in the body | Tight time |
|---|---|
| $i \leftarrow i + 1$, up to $n$ | $\Theta(n)$ |
| $i \leftarrow i + 2$, up to $n$ | $\Theta(n)$ |
| $i \leftarrow 2i$, starting at 1, up to $n$ | $\Theta(\log n)$ |
| A fixed 1001 iterations | $\Theta(1)$ |
| Two nested loops, each $n$ iterations | $\Theta(n^2)$ |
| Outer $i = 0, \ldots, n-1$; inner $j = 0, \ldots, i$ | $\Theta(n^2)$ |

The triangular case has $\sum_{i=0}^{n-1}(i+1) = n(n+1)/2$ iterations. Nested loops do not always multiply to $n^2$; use their actual bounds. The fixed-loop array example requires a large enough array for its indices to be legal.

## 5 Prefix averages: avoiding repeated work

*Source: PDF pages 70–76.* Given $X[0], \ldots, X[n-1]$, define

$$A[i] = \frac{\sum_{j=0}^{i} X[j]}{i+1}.$$

A direct algorithm recomputes each prefix sum:

```
for i = 0 to n - 1:
    s = 0
    for j = 0 to i:
        s = s + X[j]
    A[i] = s / (i + 1)
```

It performs $1 + 2 + \cdots + n = \Theta(n^2)$ additions. Maintain a running sum instead:

```
s = 0
for i = 0 to n - 1:
    s = s + X[i]
    A[i] = s / (i + 1)
```

This takes $\Theta(n)$ time. Both allocate $\Theta(n)$ output space and use $O(1)$ additional working space. Use real-number division for averages.

For $X = [5, 13, 4, 8]$, the running sums are $5, 18, 22, 30$, giving averages $5, 9, 22/3, 7.5$.

## 6 Prefix averages at geometric steps

*Source: PDF pages 77–90.* Only return averages for prefix lengths $1, 2, 4, \ldots, 2^k$, where $k = \lfloor \log_2 n \rfloor$ and $n \geq 1$:

$$A[i] = \frac{\sum_{j=0}^{2^i-1} X[j]}{2^i}, \qquad 0 \leq i \leq k.$$

Recomputing these sums costs

$$\sum_{i=0}^{k} 2^i = 2^{k+1} - 1 = \Theta(n),$$

not $\Theta(n \log n)$. A single scan with a running sum and a doubling next-prefix-length variable also takes $\Theta(n)$ time. Both approaches have $\Theta(\log n)$ output space and $O(1)$ working space. Thus avoiding recomputation here improves constants but not the asymptotic class.

```
s = 0; nextLength = 1; i = 0
for j = 0 to n - 1:
    s = s + X[j]
    if j + 1 == nextLength:
        A[i] = s / nextLength
        i = i + 1
        nextLength = 2 * nextLength
```

## 7 Mathematical toolkit

*Source: PDF pages 51–53,79,92–94.*

$$a^{b+c} = a^b a^c, \qquad (a^b)^c = a^{bc}, \qquad a^b/a^c = a^{b-c},$$
$$\log_b(xy) = \log_b x + \log_b y, \qquad \log_b(x/y) = \log_b x - \log_b y,$$
$$\log_b(x^a) = a \log_b x, \qquad \log_b x = \frac{\log_c x}{\log_c b}.$$

Here logarithm arguments are positive and bases are positive and unequal to one. $\lfloor x \rfloor$ is the largest integer at most $x$; $\lceil x \rceil$ is the smallest integer at least $x$.

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \qquad \sum_{i=0}^{n} id = d\frac{n(n+1)}{2}, \qquad \sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1} \quad (r \neq 1).$$

For fixed $r > 1$, the geometric sum is $\Theta(r^n)$; for $r = 1$ it is $n + 1$; for fixed $0 < r < 1$ it is $\Theta(1)$. The arithmetic sum is $\Theta(n^2)$ when $d > 0$ is fixed.

Optional: Stirling's approximation is $n! \sim \sqrt{2\pi n}(n/e)^n$.

**Study checklist** Be able to give explicit constants for an asymptotic proof, distinguish a loose upper bound from a tight bound, count triangular and geometric loops, and explain why running sums improve ordinary prefix averages but not geometric prefix averages.